PORNIC agglo
PAYS DE RETZ

08/02/2026

Rapport
DE STAGE

ÉTUDIANT
Nikauly De Jesus Vizcaino

FORMATION
2eme Année Bts Sio

Sommaire

Remerciements . 01

Introduction . 02

Présentation de la Collectivité . 03

Realisation du projet . 04

Développement Symfony - Authentification et Dashboard . 05

Développement Symfony - Gestion Audio et Fusion . 06

Le Moteur IA — Deep Learning et Traitement du Langage Naturel . 09

Orchestration et Mise en Production — La Maîtrise des Flux Asynchrones . 11

Conclusion . 14

Annexes . 15

Remerciements

JE TIENS À EXPRIMER MA GRATITUDE

Je tiens tout d'abord à adresser mes remerciements les plus sincères à Monsieur Frédéric Gallet, Monsieur Aymeric Bazin et Monsieur Gerald Barillet pour leur accueil, leur confiance et le temps qu'ils m'ont consacré. Leurs conseils avisés, ont été déterminants pour la réussite de ce projet.

Mes remerciements s'étendent également à l'ensemble de l'équipe de la DSI de Pornic Agglo Pays de Retz. Pour leurs disponibilité et à l'ambiance de travail bienveillante qui règne au sein du service. Cette immersion au cœur d'une collectivité territoriale a été une expérience humaine et professionnelle très enrichissante.

Enfin, je tiens à remercier Le lycée La Joliverie pour la qualité de la formation. Je pense tout particulièrement à Monsieur Cyril Contant et Monsieur Nicolas Bourgeois, dont les enseignements et le suivi pédagogique m'ont permis d'aborder ce stage avec les compétences nécessaires pour relever les défis techniques rencontrés.

Introduction

Dans le cadre de ma deuxième année de BTS Services Informatiques aux Organisations (SIO), j'ai intégré la DSI de Pornic Agglo Pays de Retz pour concevoir une plateforme de traitement audio par Intelligence Artificielle.

Le Projet : Automatisation et Sécurité

Le projet répond à une problématique métier concrète : la rédaction des comptes rendus de réunions, une tâche manuelle qui nécessite jusqu'à cinq heures de travail pour une seule heure d'audio. L'objectif était d'automatiser ce processus pour gagner en productivité, tout en respectant la confidentialité des données. En tant qu'organisme public, la collectivité impose que les données sensibles soient traitées localement sur ses propres serveurs, excluant toute utilisation d'API Cloud tierces afin de garantir une conformité totale au RGPD (Règlement général sur la protection des données).

Ma mission technique

Ma mission a consisté à développer une solution Full Stack capable de gérer l'intégralité du flux de données :

- **Interface et gestion** : Utilisation de Symfony et MariaDB pour piloter l'application.
- **Moteur d'IA local** : Intégration de WhisperX pour la transcription et de Mistral Nemo pour les synthèses et les comptes rendus, optimisés pour fonctionner sur l'infrastructure GPU.
- **Architecture** : Mise en œuvre d'un système de file d'attente asynchrone pour traiter les fichiers volumineux sans interruption de service.

Ce rapport retrace la conception de cet outil, des choix d'architecture aux défis techniques liés au déploiement de modèles d'IA en environnement souverain.

Présentation de la Collectivité

Un territoire dynamique et attractif

La Communauté d'agglomération Pornic Agglo Pays de Retz est un territoire stratégique situé entre littoral et métropoles. Elle regroupe 15 communes pour une population d'environ 70 000 habitants.
Le dynamisme démographique du territoire impose une gestion rigoureuse des infrastructures sur les 53 km de côtes et les zones rurales associées.

Missions et ressources humaines

Pour répondre aux enjeux de son territoire, Pornic Agglo Pays de Retz exerce 49 compétences structurantes.
Cette organisation repose sur une structure humaine solide composée de 270 agents permanents qui mettent en œuvre les décisions prises par les 42 élus communautaires. Une telle diversité de missions implique une activité administrative intense, marquée par de nombreuses réunions et commissions de travail. Ce volume d'échanges génère un besoin constant de production de documents officiels et de comptes-rendus, essentiels à la traçabilité de l'action publique.

La DSI mutualisée

Mon stage s'est déroulé à la Direction des Systèmes d'Information (DSI), une structure mutualisée qui gère l'infrastructure du siège et des communes membres. Intégré au pôle applicatif, ma mission consistait à apporter des competences techniques. Ce projet a été mené dans le respect rigoureux des exigences de sécurité.

Realisation du projet

RÉCUPÉRATION DES ATTENTES ET DÉFINITION DU PROJET

Le 14 janvier 2026, après une phase de tests de performance, les attentes précises du projet ont été définies pour structurer le développement:

- **Connexion LDAP** : Authentification avec les identifiants Windows via l'Active Directory (AD) pour récupérer automatiquement le nom, le prénom et l'email.
- **Interface utilisateur** : Dashboard personnalisé affichant un message de bienvenue et l'accès aux dossiers stockés en base de données.
- **Gestion des dossiers** : Création de dossiers et import de fichiers audio de tout type.
- **Traitement IA** : Possibilité pour l'utilisateur de generer une synthese et/ou un compte rendu avec Mistral Nemo et export des résultats en PDF ou DOCX.
- **Enregistrement direct** : Capture audio directement sur la plateforme et sauvegarde automatique.

VALIDATION TECHNIQUE ET INFRASTRUCTURE

Avant toute installation sur le serveur de production, une phase de validation sur machine virtuelle a été indispensable pour prendre en main l'environnement et sécuriser le déploiement futur

.

- **Environnement de test** : Installation d'une VM sous Debian 12 pour valider la stabilité du système et l'isolation des dépendances.
- **Configuration système :**
  - Mise à jour des dépôts (sources.list) pour inclure les composants non-free nécessaires aux pilotes NVIDIA.
  - Installation des dépendances critiques : **ffmpeg** pour le traitement audio et **build-essential** pour la compilation.
- **Support GPU & IA :**
  - Installation des pilotes NVIDIA et de CUDA 12.1 pour permettre l'accélération matérielle des modèles d'IA.
  - Vérification de la disponibilité du GPU via la commande **nvidia-smi** et des tests PyTorch.

CONCEPTION ET MODÉLISATION (MCD)

La conception de la base de données MariaDB assure la cohérence des informations et la traçabilité des traitements IA.

- **Entités fondamentales** :
  - **User** : Stocke les informations des agents synchronisées via l'AD.
  - **Dossier** : Regroupe les fichiers par thématique ou réunion.
  - **Fichier** : Contient les métadonnées audio, le chemin de stockage et le JSON de transcription.
  - **Synthese** & **Compte-Rendu** : Entités liées au fichier via une relation OneToOne pour stocker les productions de l'IA.
- **Logique de persistance** : Le diagramme de sequence de création de dossier **(Disponible en annexe)** montre l'interaction entre le DashboardController et la base de données pour confirmer l'ID après un SQL INSERT réussi.

Développement Symfony - Authentification et Dashboard

Cette étape visait à offrir un interface sécurisée tout en simplifiant l'accès pour les agents de la collectivité, leur permettent de s'identifier avec leurs identifiants habituel.

AUTHENTIFICATION LDAP ET PROVISIONING

J'ai implémenté le service **LdapFormAuthenticator** pour lier l'application au domaine de la collectivité. Le processus s'appuie sur un **"bind"** utilisateur vers l'Active Directory (AD). Une fois authentifié, le système effectue un **"provisioning"** : il récupère les attributs nom, prénom et mail pour créer ou mettre à jour le compte dans la base de données locale. Cela garantit que l'agent retrouve ses dossiers à chaque session.

EXPÉRIENCE UTILISATEUR (UX)

Le DashboardController centralise la gestion des espaces de travail. J'ai utilisé le moteur de template **Twig** pour personnaliser l'accueil avec les données issues de l'AD.

LOGIQUE MÉTIER

Le dashboard permet non seulement la création de dossiers via des formulaires Symfony, mais il intègre aussi un mécanisme de vérification en temps réel : à chaque chargement de page, le contrôleur vérifie si les fichiers en cours de traitement ont des fichiers de sortie **JSON** prêts pour mettre à jour les statuts automatiquement.

Voir diagrammes de Séquence 1 pour authentification Ldap en annexe
Voir diagrammes de séquence 2 pour la reation de dossier en annexe

Développement Symfony - Gestion Audio et Fusion

Le traitement de fichiers audio volumineux présentait un risque majeur de **"Timeout"** pour le serveur et le navigateur. Pour garantir la stabilité du système, j'ai mis en place une architecture asynchrone qui dissocie la réception du fichier de son analyse par l'intelligence artificielle. Ce mécanisme permet de libérer l'interface utilisateur immédiatement pendant que les calculs s'exécutent en arrière-plan sur le serveur.

STRATÉGIE DE MISE EN ATTENTE ET PERSISTANCE INITIALE

Dès que l'utilisateur valide l'envoi de ses fichiers, le système ne lance pas immédiatement la transcription. Le processus se décompose en plusieurs étapes sécurisées :

RÉCEPTION ET SÉCURISATION

Le **TranscriptionController** réceptionne les flux, génère un nom unique pour éviter toute collision et déplace les fichiers vers un répertoire temporaire sécurisé **(var/tmp/transcription/)**.

STATUT "BROUILLON"

Un premier enregistrement est créé en base de données avec le statut **BROUILLON**. Cela permet à l'utilisateur de configurer ses options (choix de la synthèse ou du compte-rendu) sans que le serveur ne commence à consommer des ressources GPU prématurément.

FUSION PRÉVENTIVE (FFMPEG)

Si l'utilisateur dépose plusieurs fichiers, le script **concat_audio.py** est appelé pour fusionner les segments. Cette opération est réalisée via FFmpeg, garantissant l'intégrité de la piste audio finale avant l'analyse par l'IA.

Voir diagrammes de séquence 3 pour l'ajout de l'audio et la concaténation

Développement Symfony - Gestion Audio et Fusion

LE QUEUESERVICE : ORDONNANCEMENT ET GESTION DU GPU

L'exécution des modèles WhisperX et Mistral Nemo nécessite une puissance de calcul GPU (VRAM) importante, qui ne peut être partagée par plusieurs processus simultanés sans risque de plantage du serveur. J'ai donc implémenté un service de file d'attente nommé QueueService :

LE MÉCANISME DE VERROU (LOCK)

Avant de lancer une tâche, le service interroge la base de données pour vérifier si un fichier possède déjà le statut **EN_COURS**. Si le système est occupé, le nouveau fichier reste au statut **EN_ATTENTE**.

EXÉCUTION EN TÂCHE DE FOND

Lorsqu'un créneau se libère, Symfony lance le script Python via la commande système nohup. Cette commande permet au script de continuer son exécution même si l'agent ferme son navigateur ou si la session PHP se termine.

Voir diagrammes de séquence 3 pour l'ajout de l'audio et la concaténation

Développement Symfony - Gestion Audio et Fusion

LE MÉCANISME DE "CALLBACK" POUR LA SYNCHRONISATION

Le défi principal de l'asynchronisme est de savoir quand le traitement est terminé pour mettre à jour l'interface utilisateur. Plutôt que de forcer Symfony à surveiller le processus en permanence, j'ai mis en place un système de rappel ou « Callback » :

APPEL INVERSÉ

À la fin de son exécution, le script Python (**transcribe.py**) utilise une commande curl pour appeler une route URL spécifique de l'application : **/internal/transcription/finished.**

MISE À JOUR ET NOTIFICATION

Cet appel "**réveille**" Symfony qui récupère alors le fichier JSON de sortie, met à jour le statut du fichier à **TERMINE** en base de données, et déclenche l'envoi d'un email de notification à l'utilisateur via le **EmailService**.

Cette architecture garantit une expérience utilisateur fluide : l'agent dépose son fichier, reçoit un accusé de réception immédiat, et peut vaquer à d'autres occupations pendant que l'infrastructure de la collectivité traite les données en arrière-plan. Elle assure également la stabilité du serveur en lissant la charge de calcul sur la carte graphique.

Voir diagrammes de séquence 3 pour l'ajout de l'audio et la concaténation

Le Moteur IA — Deep Learning et Traitement du Langage Naturel

L'intelligence de la plateforme repose sur une synergie entre deux modèles de pointe, orchestrés par des scripts Python dédiés afin d'exploiter la puissance de calcul brute du GPU. Cette architecture permet de transformer un signal acoustique complexe en un document administratif structuré, tout en garantissant la confidentialité absolue.

TRANSCRIPTION ET DIARISATION AVEC WHISPERX

Pour la phase de conversion de la parole en texte (Speech-to-Text), j'ai choisi d'implémenter **WhisperX**, une version optimisée du modèle d'OpenAI, afin de dépasser les limites de la transcription brute.

OPTIMISATION MATÉRIELLE

Le script **transcribe.py** charge le modèle **large-v3** en utilisant le moteur **faster-whisper** avec un calcul en **int8**. Ce choix technique est crucial : il permet de diviser par deux la consommation de mémoire VRAM tout en conservant une précision de transcription quasi humaine sur le GPU NVIDIA.

ALIGNEMENT ET DIARISATION

Contrairement au modèle standard, **WhisperX** intègre un pipeline d'alignement phonétique qui synchronise le texte avec l'audio au mot près. J'ai également configuré la diarisation (identification des locuteurs) via les modèles de **Pyannote**, permettant de structurer automatiquement la transcription sous forme de dialogue.

Voir diagrammes de séquence 4 pour le processus de transcription et generation de synthese et compte rendu

Le Moteur IA — Deep Learning et Traitement du Langage Naturel

SYNTHÈSE ET COMPRÉHENSION AVEC MISTRAL NEMO (12B)

La seconde étape consiste à traiter la transcription obtenue pour en extraire l'essence. J'ai sélectionné le modèle **Mistral Nemo**, développé par **Mistral AI** et **NVIDIA**, pour sa compréhension exceptionnelle des subtilités de la langue française.

PROMPT ENGINEERING

Le comportement de l'IA est piloté par des prompts système rigoureux. Pour les comptes-rendus, j'ai défini une identité d'assistant administratif expert, capable d'identifier les participants, l'ordre du jour, les points saillants et les tâches assignées de manière structurée.

STRATÉGIE DE TRAITEMENT DES DONNÉES VOLUMINEUSE

Les modèles de langage possèdent une **"fenêtre de contexte"** limitée. Pour traiter des réunions dépassant deux heures (seuil fixé à 120 000 caractères), j'ai développé une logique de découpage et fusion **(Map-Reduce)**. Le script **generer_synthese.py** découpe la transcription en blocs de 60 000 caractères, extrait les informations clés de chaque bloc, puis réalise une synthèse finale consolidée en conservant l'attribution des paroles à chaque intervenant.

Cette approche hybride garantit que, quelle que soit la durée de la réunion, l'agent reçoit un document cohérent, fidèle à la transcription originale et respectant les codes de la rédaction publique.

Voir diagrammes de séquence 4 pour le processus de transcription et generation de synthse et compte rendu

Orchestration et Mise en Production — La Maîtrise des Flux Asynchrones

Le défi majeur de l'architecture résidait dans la synchronisation entre l'interface web (**PHP**), fonctionnant de manière synchrone, et les moteurs d'IA (**Python**), dont les temps de traitement sont longs et asynchrones. Cette "rupture" technologique a nécessité la mise en place d'un chef d'orchestre capable de gérer les ressources matérielles et la communication bidirectionnelle entre les services.

GESTION DE LA FILE D'ATTENTE (QUEUESERVICE)

Pour garantir la stabilité du serveur, il était impossible de laisser plusieurs transcriptions s'exécuter simultanément, au risque de saturer la mémoire vidéo (VRAM) du GPU et de provoquer un plantage système.

LE MÉCANISME DE SÉMAPHORE

J'ai développé le **QueueService** qui agit comme un verrou logique. Avant chaque lancement, le service vérifie en base de données si un autre fichier possède le statut **EN_COURS**. Si le GPU est occupé, le fichier est placé en **EN_ATTENTE**.

L'EXÉCUTION VIA NOHUP

Pour détacher le processus Python du cycle de vie de la requête HTTP, j'utilise la commande système **nohup**. Cela permet au script **transcribe.py** de continuer son exécution en arrière-plan même si l'utilisateur ferme sa session web.

Voir diagrammes de séquence 4 pour le processus de transcription et generation de synthse et compte rendu

Orchestration et Mise en Production — La Maîtrise des Flux Asynchrones

EXPORTATION MULTIFORMAT ET SÉCURISATION

L'application doit fournir des documents directement exploitables dans le cadre administratif.

MISE EN PAGE PROFESSIONNELLE

Le **CompteRenduController** ainsi que le **SyntheseController** et le **TranscriptionController** pilotent l'exportation. J'ai intégré la librairie **Dompdf** pour la génération de PDF, avec une injection de feuilles de style CSS. Pour les besoins de réécriture, j'ai implémenté **PhpWord**, permettant de générer des fichiers .docx. Une fonction de nettoyage automatique **(deleteFileAfterSend)** supprime les fichiers temporaires après le téléchargement pour libérer l'espace disque.

Voir diagrammes de séquence 5 pour le processus de telechargement

Orchestration et Mise en Production — La Maîtrise des Flux Asynchrones

DÉPLOIEMENT EN PRODUCTION ET SÉCURISATION HTTPS

La mise en service a impliqué la configuration avancée du serveur web **Apache2** pour assurer la confidentialité des échanges, notamment lors de la transmission des identifiants Active Directory et des fichiers audio.

SÉCURISATION HTTPS ET CONFIDENTIALITÉ

J'ai configuré le **VirtualHost** Apache pour le domaine papyrus.pornicagglo.fr. Le chiffrement est imposé par une redirection systématique du port 80 vers le 443 et l'activation du moteur **SSLEngine** avec les certificats SSL Wildcard de la collectivité. Cette configuration garantit l'intégrité des fichiers audio et la sécurité des identifiants transitant lors de l'authentification.

RESTRICTION D'ACCÈS ET FILTRAGE IP

Afin de réduire la surface d'attaque, j'ai durci la configuration de **phpMyAdmin**. L'accès à l'interface de gestion de la base **MariaDB** est désormais restreint par une directive **Require ip** dans le fichier **phpmyadmin.conf**, limitant les connexions. Toute tentative extérieure est rejetée par défaut.

DURCISSEMENT SYSTÈME (UFW)

Le pare-feu du serveur **Debian 12** a été configuré pour verrouiller l'intégralité des ports, à l'exception du port **443 (TCP)**. Ce cloisonnement strict assure que seul le trafic web sécurisé est autorisé à atteindre l'application

Voir annexes 6 et 7 pour la configuration des fichiers phpmyadmin.conf et papyrus.conf

Conclusion

Ce stage au sein de la DSI de Pornic Agglo pays de retz a été un étape clé de mon BTS SIO. J'ai pu mener a bien le projet "Papyrus", de la validation sur VM Debian 12 jusqu'au déploiement en production sur serveur physique. Cette expérience a consolidé ma maîtrise de Symfony et m'as permi d'intégrer des technologies innovantes comme WhisperX et Mistral Nemo

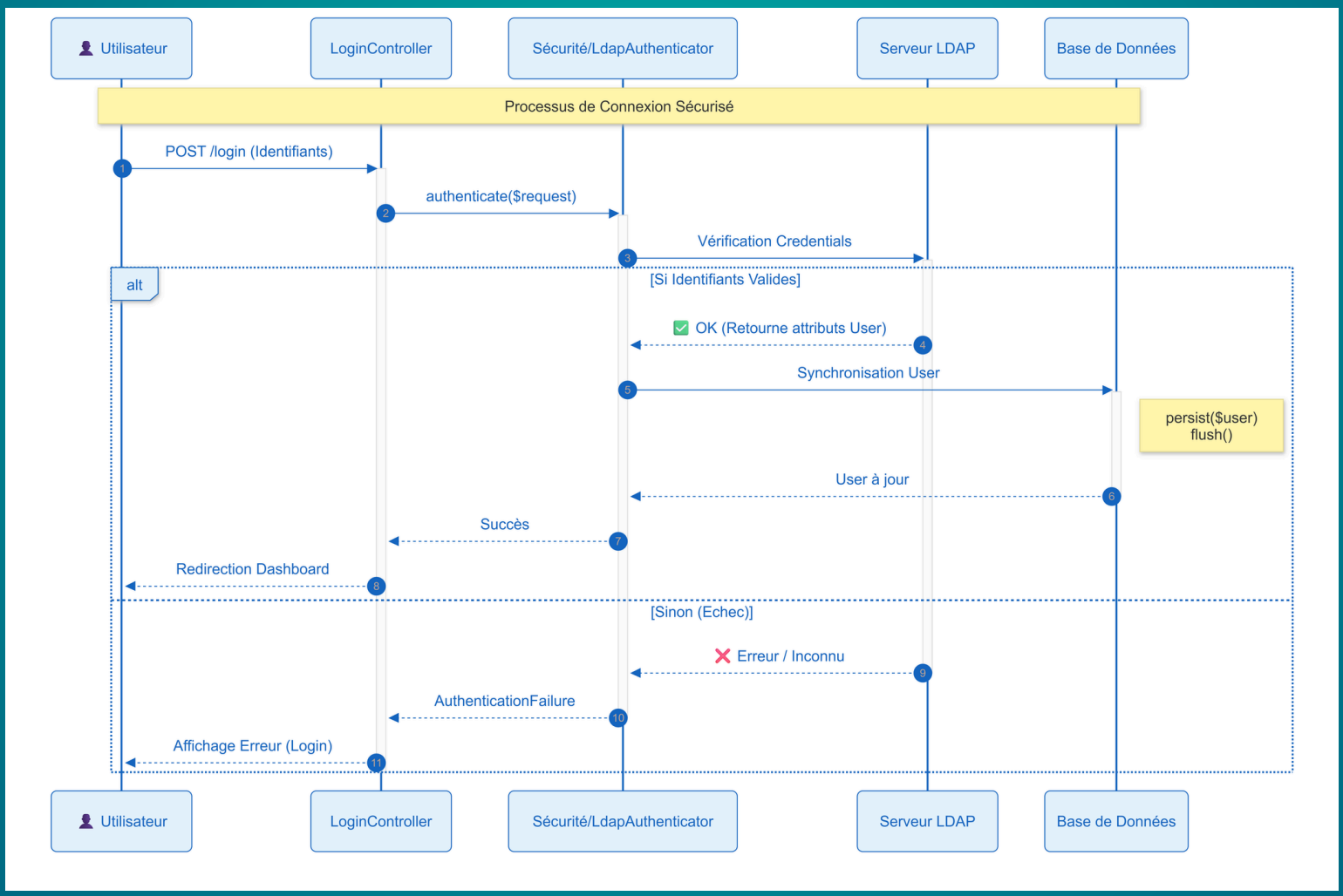
Sur le plan technique, j'ai relevé les defis de l'asynchronisme te de l'optimisation des ressources GPU via le "QueueService", garantisant la stabilité lors du traitement de gros volumes de données. Ce projet m'a aussi sensibilisé aus enjeux de la sourverianeté numerique et du respect du RGPD en collectivité, imposant un traitement des données exclusivement local pour sécuriser les échanges administratif.

La réussite de ce projet confirme mon souhait de poursuivre mes etudés en L3 MIAGE. Ce stage m'as montré l'importance de concilier le développement technique avec les besoin de gestion d'une organisation. La licence MIAGE me permettra d'approfondir cette double compétence, indispensable pour piloter des projets de transformation numérique complexes.

Annexes

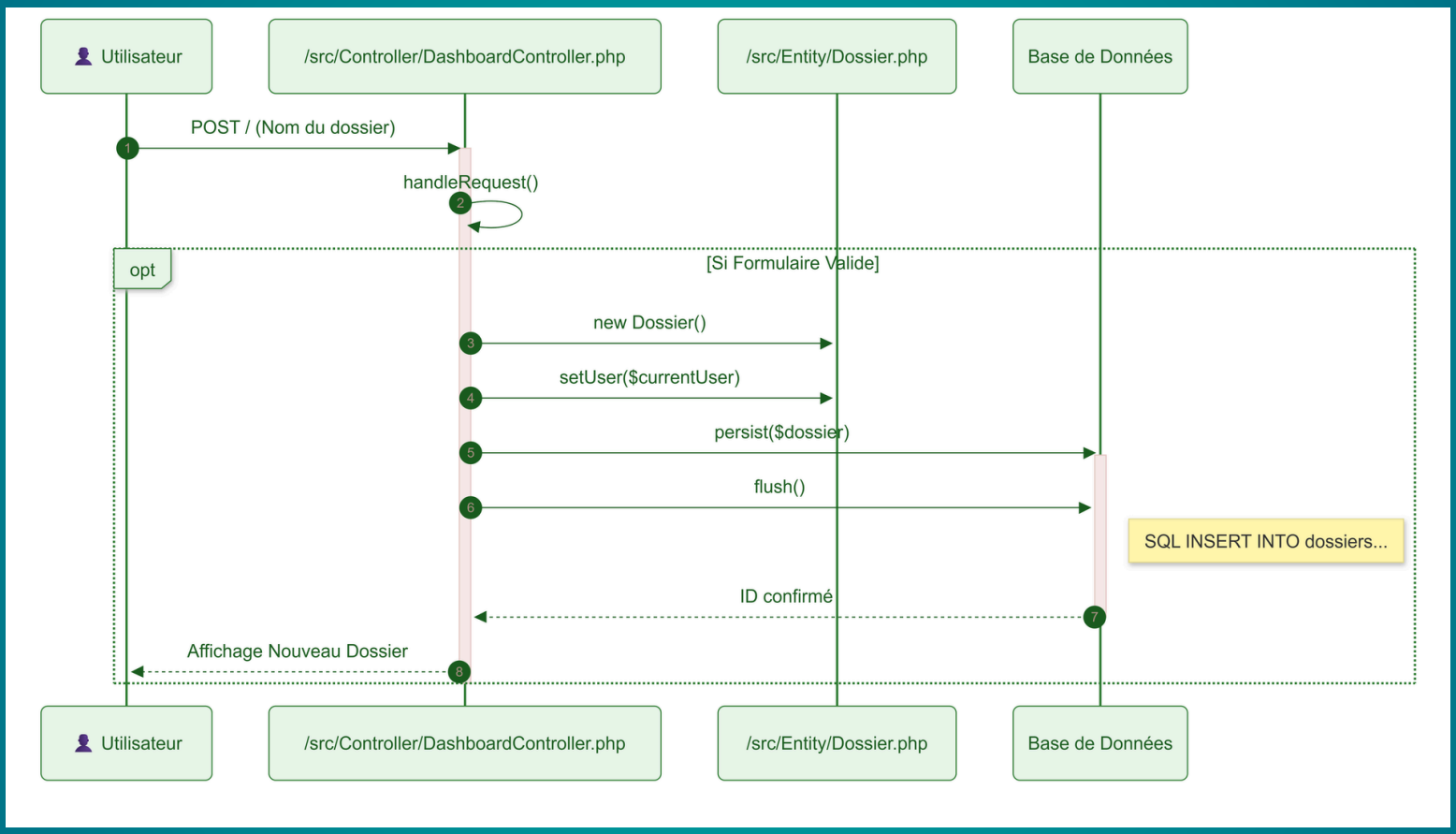
Annexes

diagrammes de séquence 1



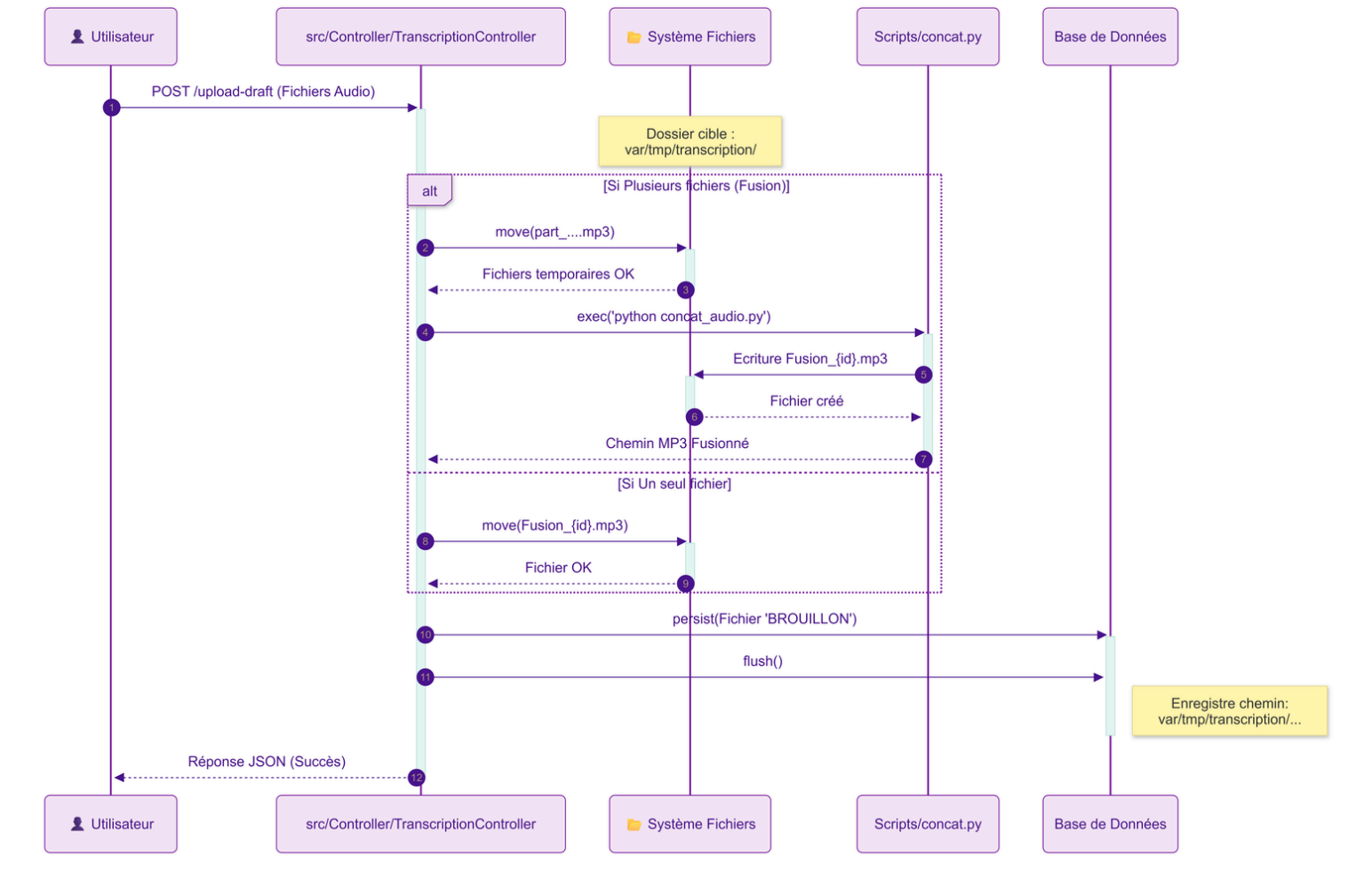
Annexes

diagrammes de séquence 2



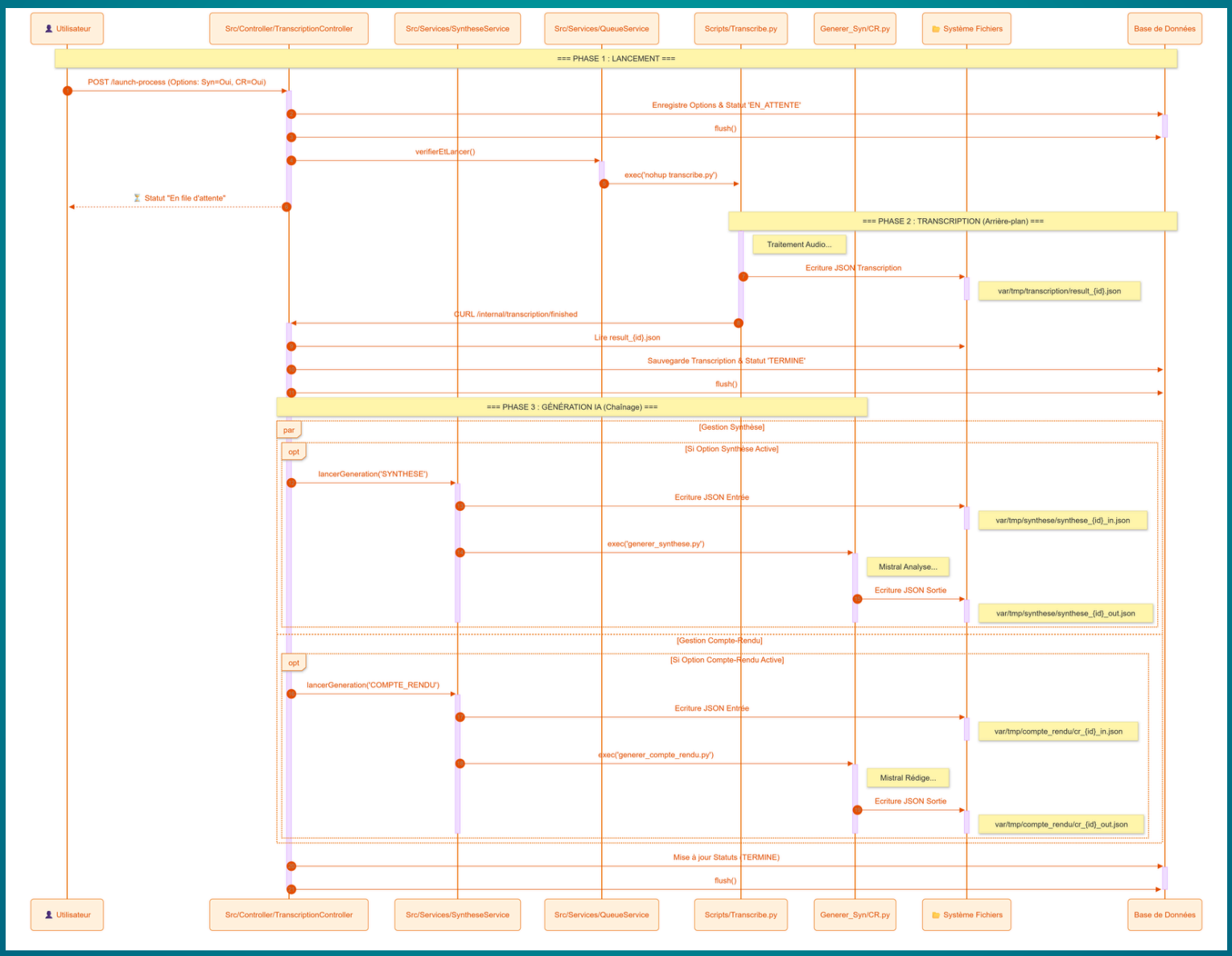
Annexes

diagrammes de séquence 3



Annexes

diagrammes de séquence 4



Annexes

phpmyadmin.conf

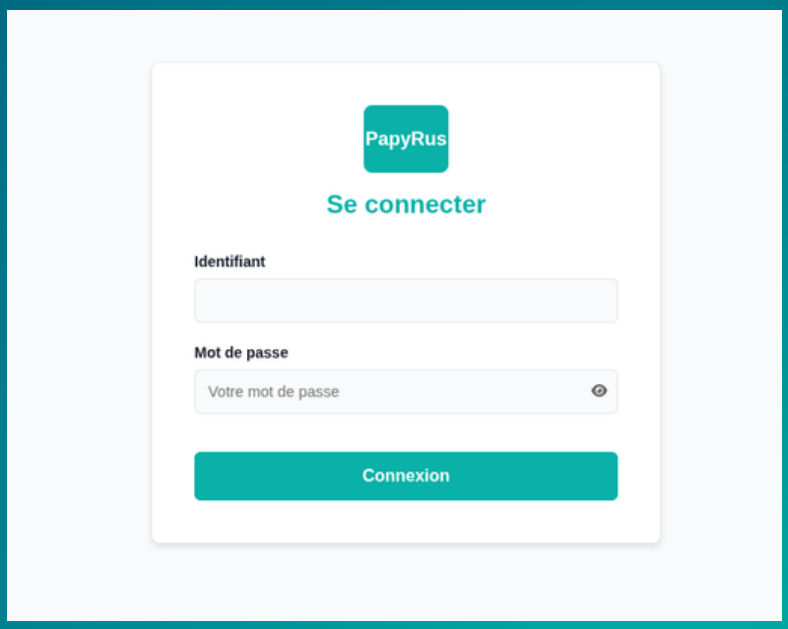
```
<RequireAny>
    On refuse tout le monde
    Require all denied

    Require ip 10.40.1.210
    Require ip 10.40.1.210
</RequireAny>
```

papyrus.conf

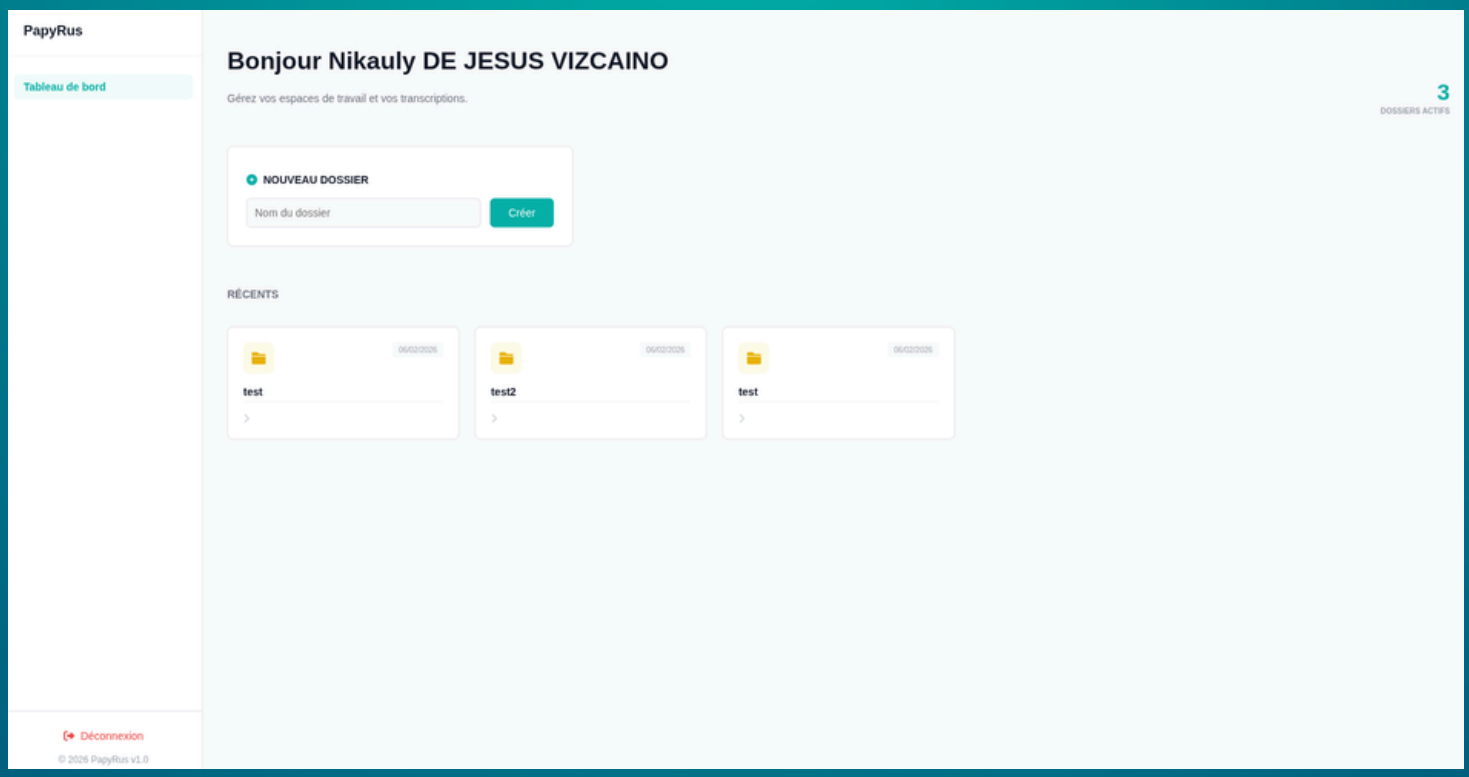
```
<VirtualHost *:80>
    ServerName papyrus.pornicagglo.fr
    Redirect permanent / https://papyrus.pornicagglo.fr/
</VirtualHost>


<VirtualHost *:443>
    ServerName papyrus.pornicagglo.fr
    DocumentRoot /var/www/papyrus/public

    SSLEngine on
    SSLCertificateFile    /var/www/wildcard/ssl.crt/wilcard_cert.crt
    SSLCertificateKeyFile /var/www/wildcard/ssl.key/wildcard_key.key
    SSLCertificateChainFile /var/www/wildcard/ssl/wildcardinter_pornic.crt
    SSLCertificateChainFile /var/www/wildcard/ssl/wildcardinter_pornic.crt

    <Directory /var/www/papyrus/public>
        AllowOverride All
        Require all granted
        FallbackResource /index.php
    </Directory>
</VirtualHost>
```

Annexes

page de connection

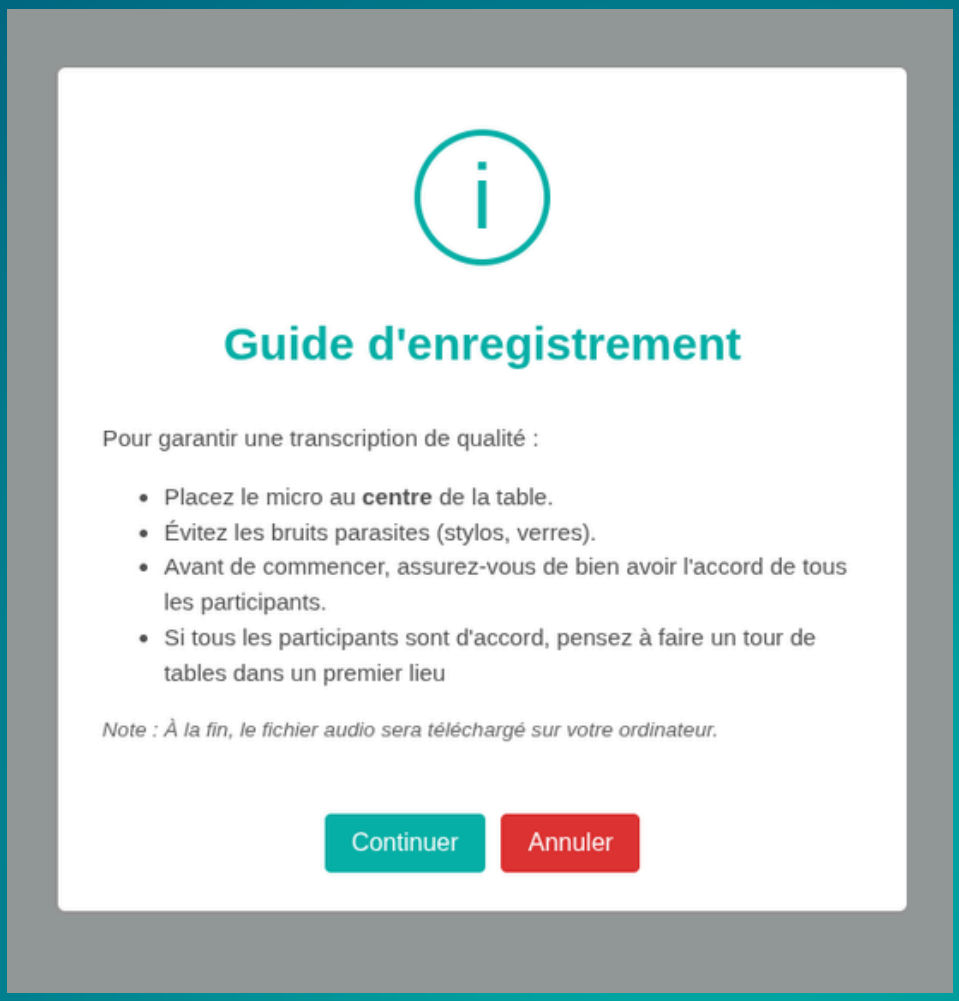


Dashboard

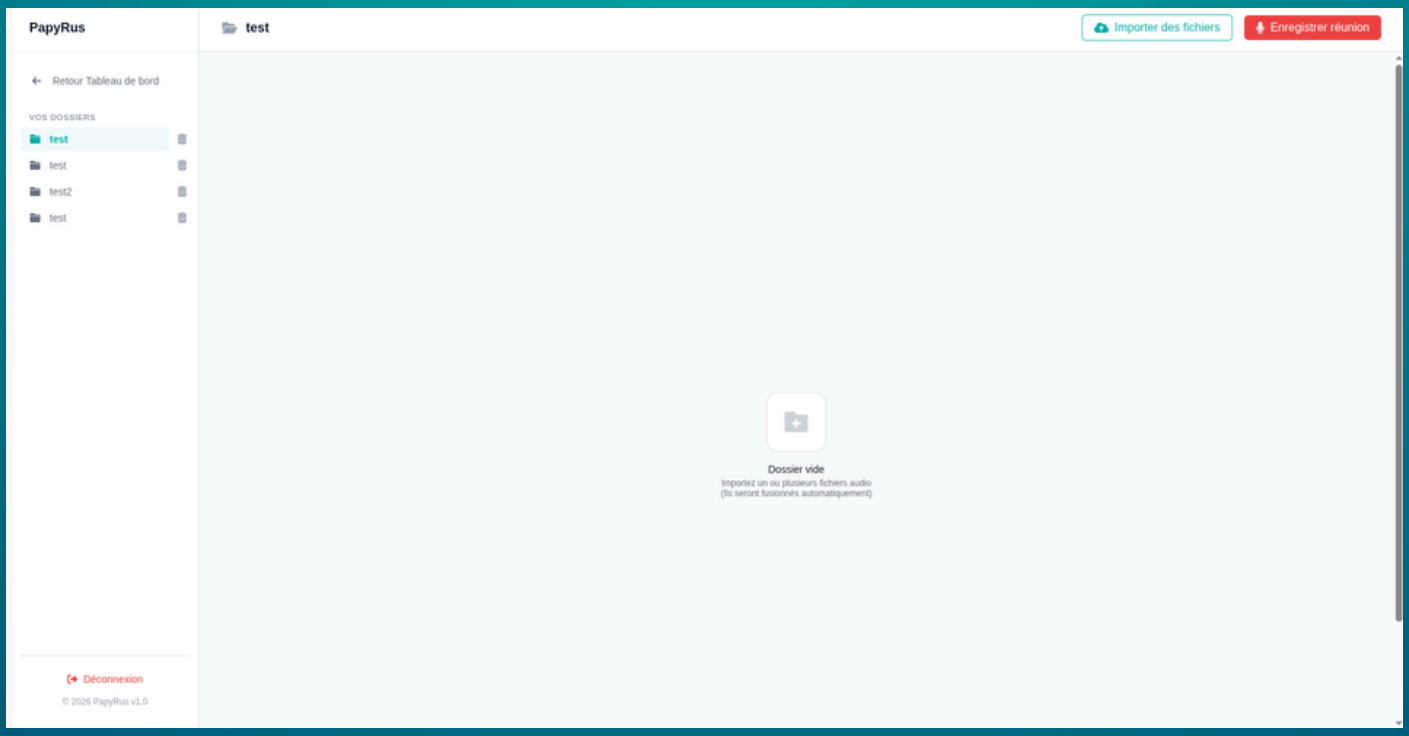


Annexes

pop up guide d'enregistrement

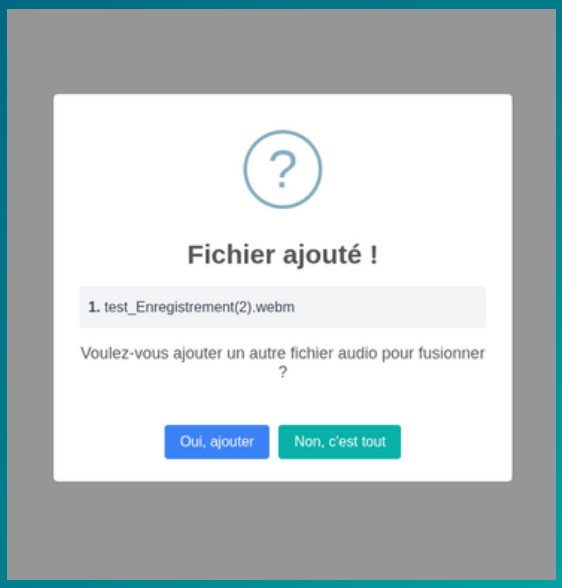


page dossier

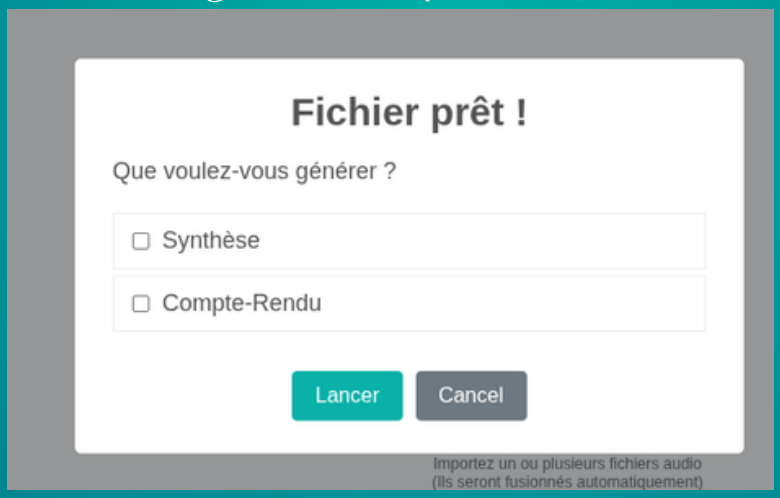
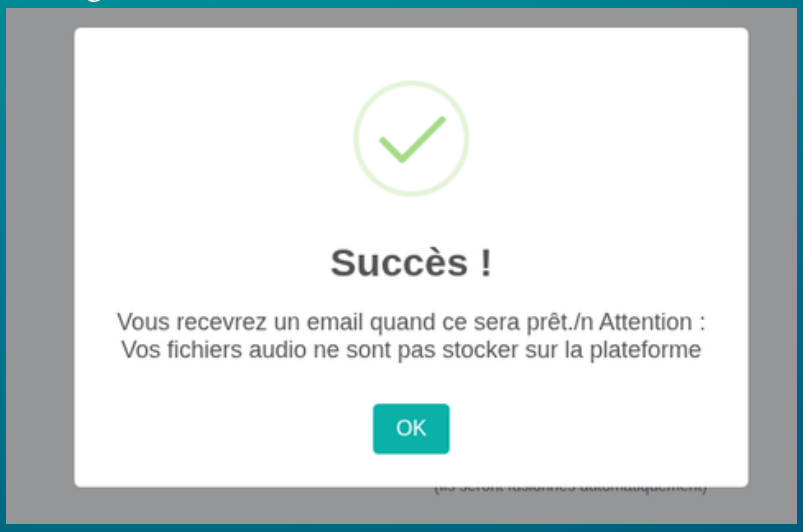


Annexes

pop up import fichier audio



choix synthese/compte rendu



confirmation